ST. XAVIER'S COLLEGE

(Affiliated to Tribhuvan University)

Maitighar, Kathmandu

Project Report on the Title

"Fitaura: A Fitness Tracker"

Submitted By:

Riwaz Mahat (024BSCIT032)
Shreyana Adhikari (024BSCIT037)
Sulav Pandey (024BSCIT043)

Maitighar, Kathmandu

Submitted To:

Mr. Lalit Raj Giri
Department of Computer Science

August 2025

Student Declaration

This is to confirm that we have successfully completed our case study/project titled "Fitaura: A Fitness Tracker" under the guidance of Mr. Lalit Raj Giri. This work was carried out as part of the academic requirements for the **Bachelor's degree in Computer Science and Information Technology (B.Sc.CSIT)** at the Institute of Science and Technology (IOST), Tribhuvan University. The project is entirely our own, and it has not been submitted anywhere else before.

Riwaz Mahat (024BSCIT032)

Shreyana Adhikari (024BSCIT037)

Sulav Pandey (024BSCIT043)

Acknowledgement

We want to take a moment to thank everyone who helped us along the way during this project. It honestly wouldn't have been possible without the support, encouragement, and guidance we received from so many amazing people.

We are very thankful to our supervisor, **Mr. Lalit Raj Giri.** He guided us through every step with patience, gave us honest feedback, and supported us even when things got tough.

A heartfelt thank you to **Mr. Ganesh Yogi**, our Head of Department. We're genuinely thankful for the time and energy he put into guiding us in our academics. We're especially thankful to **Rev. Father Principal Dr. Augustine Thomas, S.J.**, whose leadership has always been an inspiration.

A big thank you to **St. Xavier's College, Department of Computer Science**, for giving us the chance to work on this summer project. We really appreciate the trust the college placed in us, it pushed us to give it our best.

And of course, we couldn't have done this without the love and support of our friends, families, and everyone who cheered us on from the side-lines. Your belief in us kept us going, even on the days it felt overwhelming.

Riwaz Mahat (024BSCIT032)
Shreyana Adhikari (024BSCIT037)
Sulav Pandey (024BSCIT043)

11th August, 2025

Abstract

Fitaura is a unique and seamless fitness tracker application that is based on C++, machine learning and artificial intelligence to provide the user with predictive fitness information. The important parameters of fitness measured by the application within a 7-day cycle are daily steps, daily calories intake, sleep hours, consumed water, height, weight, and BMI calculation. In such data, Fitaura will use a linear regression model in Python to foretell the following 15 days of fitness data to help users beforehand foresee patterns and change their routinely schedules.

Fitaura architecture is modular; thus, it is more scalable and reusable. It implements major object-oriented programming principles which include inheritance, encapsulation, and polymorphism. Inheritance enables building of specialized classes that helps in reusing codes and encapsulation shields the internal state of things thus offers better handling of data. Polymorphism allows a flexible way in terms of interacting with the objects and this is a key requirement to a dynamic application. The backbone behind the machine learning implementation is the C++ which facilitates effective processing and prediction in real time. With this high-performance backend, Fitaura will be capable of producing on-time and precise fitness predictions based on captured data.

The GUI is written based on the Qt toolbox and Qt Widgets with a convenient interface. Such characteristics as monitoring the progress with the use of visual graphs and personalized recommendations depending on predictive analytics are the significant features that allow a user to make an informed decision regarding their fitness journeys. The modularity of Fitaura also simplifies the process of adding new features, e.g., new fitness metrics, and better models trained by machine learning. By and large, Fitaura is a complete fitness tracking tool which enables users to put themselves in control of their health without having their heads to the wayside due to changes in technology.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

OOP	Object Oriented Programming
GUI	Graphical User Interface
ID	Identification
BMI	Body Mass Index
AI	Artificial Intelligence
ML	Machine Learning
CLI	Command-line Interface
UI	User Interface
GIF	Graphics Interchange Format
I/O	Input/Output
RAM	Random Access Memory
GB	Gigabyte
OS	Operating System
MSVC	Microsoft Visual C++
JSON	JavaScript Object Notation
CSV	Comma-Separated Values

Chapter I: Introduction

1.1 Background of the Project

Personal health and fitness have also become a crucial issue in the recent years and recently in particular due to the global pandemic. As individuals are getting more aware of their lifestyle choices, increased demands of digital tools that help and support physical well-being in people by tracking it regularly, providing feedback, and analysing the acquired data in a meaningful way.

Being computer science students, we were aware of the possibility of integrating the concepts of Object-Oriented Programming (OOP) with the current development frameworks to provide a solution to this necessity. It led us to develop Fitaura a fitness tracking app that does not just store the data that users provide but can also help them by creating a time series model that analyses the data and forecasts fitness data going forward.

The incentive to create this project was two separate reasons:

- To show our knowledge on C++ programming with actual implementation with the use of Qt creator as the development platform.
- To incorporate machine learning to get predictive insight, thus providing more value to the app than merely logging data.

Fitaura is developed as a cross-platform desktop program that is primarily programmed in C++ on the Qt framework. It offers the responsive and interactive user interface that is developed with Qt Widgets, whereas the backend implementation is taken with the help of OOP-driven C++ classes. Python-enabled machine learning model is added to the system to predict fitness metrics depending on the history of a particular user. This forecasting component motivates users to remain regular with their training regimes and keep track of the trends in the long run.

In general, Fitaura is rather an exhibition piece and a handy device at the same time, as it combines an idea of a structured programming, user-interface along with machine learning advancements altogether and creating one whole and working system. It expresses our fascination with software development, data science, and user-centred design as well as approaches a timely and real-life issue; being healthy and informed in digital age.

1.2 Problem Statement

Tracking your day-to-day healthy habits is often a difficult and should be not a manual entry to a device, instead an availability of a fitness tracker that records, predicts, analyses and summarises the daily workout regimen, habits and other health related factors is the future of fitness in modern day. Most applications focus only on historical data rather than predictive insights.

There is a need for an AI-assisted system that can not only track fitness parameters but also predict changes and offer guidance. Our project addresses this by integrating a machine learning model into a user-friendly C++/Qt-based fitness tracker, with an interactive Graphical User Interface (GUI), based on Qt Widgets. The assistance provided by the tracker can overlook the downside and the upside of the user and provide them with insights on how to improve their health.

1.3 Objectives

The major objective of this project is to create and successfully operate Fitaura, an AI-enabled fitness tracking system, that utilizes the object-oriented features of C++ along with Qt Widgets for GUI development and integrates a Python-based machine learning model for predictive analysis of health metrics. The vital regions that showcase the complexity of the tracker built using the concept of object-oriented programming is demonstrated with these objectives:

1.3.1 To Demonstrate Object-Oriented Programming (OOP) Concepts in C++

The project serves as a practical implementation of key object-oriented programming concepts:

- **Encapsulation**: All fitness data for users is securely managed within class structures (e.g., user, appcontroller, fitnesslog).
- **Inheritance**: The concept of inheritance is used to relate one class with another class, as our tracker's behaviour depends upon the separate data members and their co-relationship with each other to function.
- **Polymorphism**: In Fitaura, runtime polymorphism is used to manage different behaviours for different types of system components.
- **Abstraction**: Complex internal logic such as ML integration and fitness scoring is hidden from the user and encapsulated in high-level functions.

1.3.2 To Collect and Manage Multi-Parameter Fitness Data

The system captures a broad range of health and lifestyle inputs, including:

- **Static details**: User ID, date, age, gender, height, and weight, sleep hours, daily water and calories intake, daily steps and calculation of BMI.
- **Dynamic metrics over 7 days**: Daily steps, calorie intake, water intake, sleep hours, and calculated BMI are calculated, observed and taken in account to provide a prediction and report summary for the user.

1.3.3 To Develop a GUI-Based Fitness Tracker Using Qt and Qt Widgets

The aim is to provide a user-friendly interface that mirrors modern fitness applications:

- Users can log in and view their daily data and weekly reports with charts showing trends across all health parameters.
- The GUI is built using Qt Widgets, ensuring flexibility of running the tracker on all platforms for clarity and ease of use.
- The Qt backend communicates well with the GUI and ML components to provide near real-time interaction.

1.3.4 To Integrate a Python-Based Machine Learning Based on Linear Regression Model

A linear regression model, trained using dummy datasets in Python, was integrated with the C++ backend to:

- Forecast trends in each of the 11 parameters for the next 15 days.
- Provide quantitative and attributive predictions on whether metrics are increasing, decreasing, or remaining stable and what needs change in the records.
- To calculate and summarize an overall fitness score for each user based on the predicted changes.

1.3.5 To Provide Predictive Feedback and Insightful Reporting

- Analyses trends in user health habits based on recorded data.
- Generates personalized summaries with recommendations for everyone.
- Shows visual cues (e.g., fitness levels like Excellent, Good, Average and Poor) to help users understand their trajectory and their fitness performance.
- Stores or exports reports for future reference.

1.3.6 For Free Assistance for Those Who Can't Afford Professional and Premium Fitness Plans

- To uplift the fitness community with easy availability of our tracker to help people easily achieve their fitness goals.
- To make sure people who hesitate to upgrade to a premium fitness planner or even a fitness trainer can benefit from our tracker.

1.4 Scope and Limitations

1.4.1 Scopes

The system currently supports up to 15 users and tracks 11 key fitness parameters, offering basic functionality for health data monitoring and prediction. It demonstrates the use of object-oriented programming in C++ along with the integration of a simple linear regression model for forecasting future fitness trends. While the current version is limited in scale, it is built with modularity and scalability in mind, making it adaptable for future enhancements such as real user data collection, integration of wearable sensors, advanced machine learning models, and cloud-based data storage.

1.4.2 Limitations

At present, the system relies on dummy training data, which limits the accuracy and practical relevance of its predictions. The machine learning component is restricted to a basic linear regression model, which may not capture complex user behaviour or health patterns. Additionally, the system supports only 15 users and requires manual data entry, lacking real-time input from sensors or fitness devices. It also does not include cloud connectivity or user authentication, making it unsuitable for large-scale or real-world deployment at this stage.

Chapter II: Review of Related Work and Literature

2.1 Background

Commercial fitness-related platforms like Fitbit, Apple Health, Google Fit, Samsung Health, Nike Training Club and more allow users to monitor various health metrics such as daily steps, heart rate, sleep patterns, calorie intake, and physical activity. These platforms gather real-time data using wearable sensors and smartphones, wrist watches, heartbeat monitoring devices helping users to track their fitness routine and stay informed about their lifestyle habits.

However, most of these platforms are focused on data logging and visualization rather than prediction. While they surely provide trends, comparisons, and summary, they rarely integrate machine learning models to give future forecasts and personal recommendations based on user data history. It brings predictive intelligence to personal health in a competitive environment, making it more accessible and customizable.

2.2 Related Work (Literature Review)

2.2.1 Web-Based Fitness Management Systems

Patel and Shah (2019) developed a fitness management system designed to help users record and monitor their exercise routines, diet plans, and overall progress through a web platform. Their system allowed multiple users to maintain profiles and input health data manually, focusing on structured data management and user-specific progress tracking. However, Fitaura advances this concept by integrating a predictive model that forecasts future fitness trends based on existing data, offering users proactive insights beyond mere historical tracking.

2.2.2 Online Health Monitoring Portals

Singh et al. (2020) proposed an online health monitoring portal that collected user health metrics via web input forms and provided immediate feedback and personalized recommendations. While Fitaura also collects user health parameters, it extends the concept by incorporating a machine learning prediction module that not only reflects current data but also projects future fitness outcomes, enabling users to better plan their routines.

2.2.3 Cloud-Based Personal Fitness Platforms

Kumar and Mehta (2021) focused on creating a platform that allowed users to monitor their fitness data remotely, enabling data synchronization across sessions and devices. Fitaura adopts a similar multi-user data handling approach but uniquely combines this with a backend predictive analytics component, which currently predicts fitness scores over a future time window, enhancing the application’s value as a decision-support tool.

2.2.4 Hybrid Web and CLI Fitness Applications

Patel and Gomez (2020) investigated systems that combined web interfaces for ease of use with command-line interfaces (CLI) preferred by technical users. Their study highlighted the benefits of dual-interface systems in increasing accessibility and functionality across user groups. Fitaura is architected with modularity in mind, aiming to eventually support both graphical and command-line interfaces to cater to diverse user preferences while maintaining synchronized data and consistent functionality.

Table 2.1: Summary of Related Work & Fitaura's Contributions

Related Works	Similarities	Fitaura's Contribution
Patel & Shah (2019)	Focuses on tracking fitness for multiple users	Adds machine learning prediction for future fitness trends
Singh et al. (2020)	Emphasizes user data input along with feedback	Incorporates forecasting of fitness outcomes
Patel & Gomez (2020)	Offers dual-interface accessibility	Modular architecture with Qt Widgets GUI and CLI interface planned
Kumar & Mehta (2021)	Addresses management of data for multiple users	Combines synchronization with predictive modelling

This project enhances existing fitness systems by integrating multi-user management, predictive analytics, and synchronized data handling within a user-friendly modular design. With a Qt Widgets GUI and planned CLI support, it provides a flexible and comprehensive solution for diverse users.

Chapter III: Methodology

3.1 System Overview

Fitaura, is a fitness tracking system is a modular application developed in C++ that integrates a Qt Widgets graphical user interface (GUI) and plans for a command-line interface (CLI) to enhance accessibility. It allows users to register, input and track multiple fitness parameters, and receive predictive analytics on their fitness progress based on historical data. Emphasizing object-oriented design and data persistence via structured file storage, the system is designed for both practical fitness tracking and academic demonstration of software engineering principles.

3.2 Major System Implementation Diagrams

3.2.1 Data Flow Diagram (DFD)

The DFD represents the flow of information between users, interfaces, backend logic, and data storage.

Key Processes:

- **User Interfaces:**
  - Qt Widgets GUI for everyday users providing intuitive data entry and visualization.
  - Planned CLI for technical users and administrators for advanced control.
- **Backend:**
  - C++ core managing data validation, business logic, and predictive modelling.
- **Data Storage:**
  - Structured text files (pipe-separated) for maintaining user profiles, fitness parameters, and prediction data.

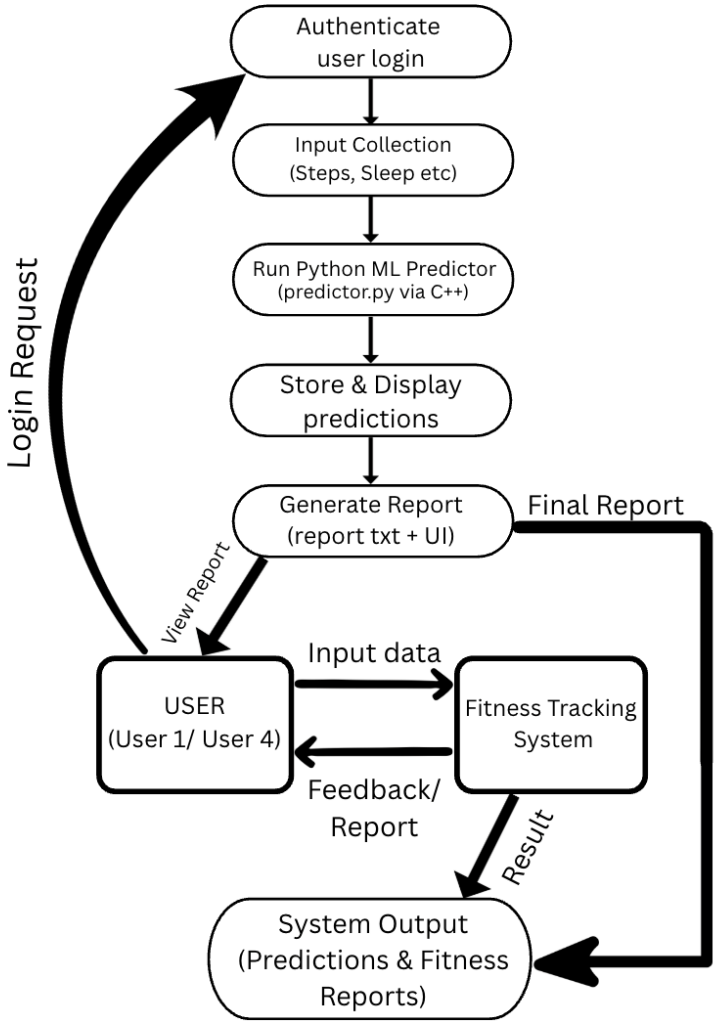


Figure 3.2.1: Dataflow diagram

Main Operations Modelled:

- User registration and authentication with encrypted password handling.
- Fitness parameter entry, update, and retrieval.
- Predictive fitness scoring using a linear regression model.
- Data synchronization ensuring consistency between interfaces.

3.2.2 Class Diagram

The Fitness Tracker system's object-oriented components are structurally summarized in the Class Diagram.

i. User Class

Attributes:

- name: string — Stores the name of the user.
- age: int — Stores the age of the user.
- gender: string — Represents the gender.
- height_cm: float — User's height in centimetres.
- weight_kg: float — User's weight in kilograms.
- activity_level: string — Activity description
- username: string — Login username.

Methods:

- getters and setters — For all personal data fields.
- calculateBMI() — Returns the user's BMI value.
- displayUserInfo() — Outputs formatted user information.

ii. Fitness Data Class

Attributes:

- steps: int — Number of steps walked.
- calories: float — Calories burned.
- sleep_hours: float — Sleep duration.
- water_intake_l: float — Water consumed (in litres).

Methods:

- calculateScores() — Calculates scores based on fitness parameters.
- getRecommendations() — Returns feedback based on scores.
- getters and setters — For all fitness metrics.

iii. Fitness Report Class

Attributes:

- summary: string — Formatted result to be shown in the GUI and saved to file.
- scores: map<string, float> — Stores category-wise fitness scores.
- recommendations: vector<string> — Personalized health advice.

Methods:

- generateReport() — Combines scores and recommendations into a structured report.
- saveReport() — Writes the report to outputs/report.txt.

iv. ML Interface Class (Python Integration)

Attributes:

- scriptPath: string — Absolute path to predictor.py.
- outputPath: string — Path to the prediction and report output files.

Methods:

- runMLScript() — Executes Python script via QProcess.
- readReport() — Reads the generated report.txt.
- parsePredictions() — Extracts future predictions from predictions.json.

v. Main Window Class (Qt Widgets GUI)

Attributes:

- GUI elements: QLabel, QPushButton, QLineEdit, etc.
- Input forms: Fitness data input widgets.
- Output panels: Displays full fitness report.

Methods:

- initUI() — Sets up all widgets and layout.
- loadImages() — Loads images into labels (e.g., exercise.png, heartbeat.gif).
- handleLogin() — Verifies allowed usernames.
- triggerPrediction() — Calls ML backend and displays result.
- displayReport() — Shows formatted result in GUI.

Relationships and Interactions

Inheritance:

- MainWindow inherits from QMainWindow (Qt base class).

Associations:

- User and FitnessData are associated — each user inputs corresponding data.
- FitnessData is processed by predictor.py via the MLInterface.
- FitnessReport uses prediction output to build the report.
- MainWindow manages the overall UI and connects to all backend modules.

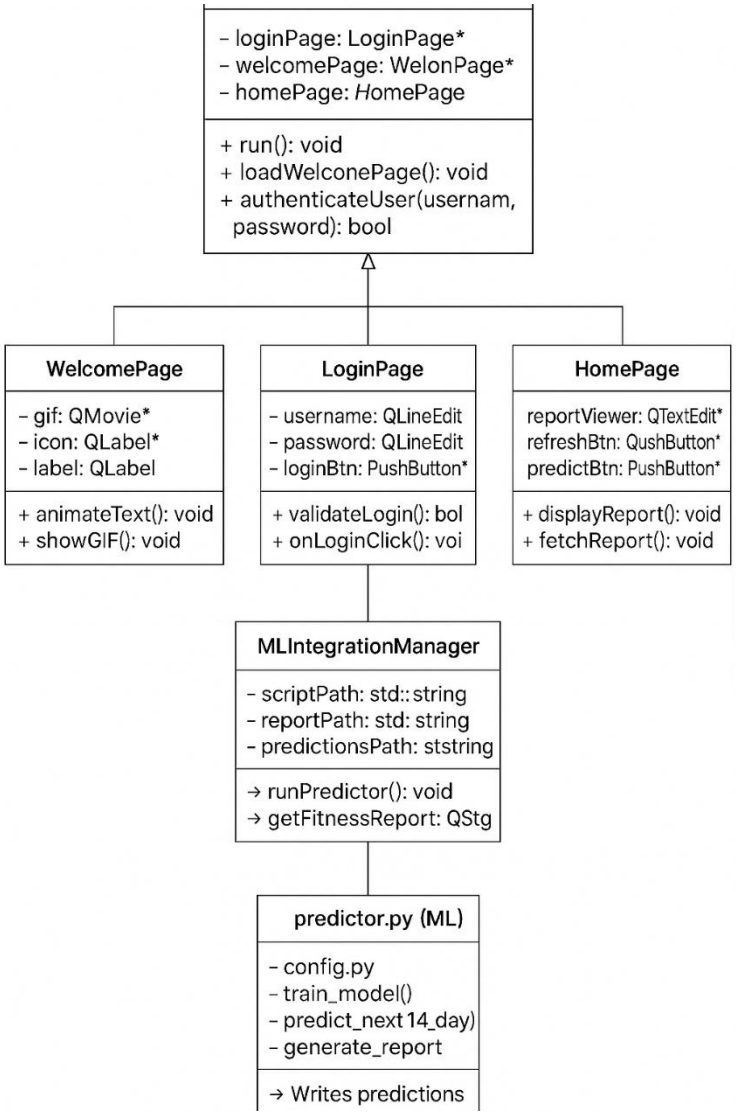


Figure 3.2.2: Class Diagram

3.3 Features of the System

The Fitness Tracker system offers a set of robust and interactive features for monitoring and enhancing the physical health of users. It utilizes a combination of object-oriented C++ backend logic, machine learning integration via Python, and an animated Qt Widgets GUI. The features are classified into different modules as described below:

3.3.1 User Authentication

- A simple login system is implemented where only predefined users (e.g., “User1” and “User4”) are allowed to access the application.
- This layer ensures that the system is used only by authorized users for evaluation and demonstration purposes.

3.3.2 Graphical User Interface (GUI)

- Developed using Qt Widgets, it boasts a professional visual appeal that enhances user experience and ensures a polished, modern interface.
- The Welcome Page features:
    - A white background with animated "WELCOME" text.
    - GIF-based heartbeat icon and static exercise icon displayed beside the welcome text.
- The Home Page includes:
    - Sidebar with navigational icons (Home, Summary, Settings, Notification).
    - Display of steps, calories, water intake, and sleep using custom-designed QLabel-based stat boxes.
- The Summary Page displays:
    - Predicted fitness progress using charts and an automatically generated textual report.

3.3.3 Data Entry and Validation

- Users can input daily fitness metrics such as:
    - Height
    - Weight
    - Number of steps walked
    - Sleep hours
    - Water intake (in litres)
    - Calories burned
- All fields are validated to ensure only numerical inputs are accepted.

3.3.4 Backend C++ Logic

- Backend logic is written in C++ using object-oriented programming.
- Classes like User, FitnessData, and FitnessReport handle data modeling, score calculations, and file outputs.
- The FitnessReport class generates structured feedback and recommendations based on the inputs provided.

3.3.5 Machine Learning Prediction (Python Integration)

- A Python script (predictor.py) is executed from the C++ backend using QProcess.
- The script uses a **scikit-learn pipeline** to perform 15-day fitness forecasting on:
    - Steps
    - Calories
    - Water intake
    - Sleep hours

- Predictions are saved in outputs/predictions.json, and a full report is written to outputs/report.txt.

3.3.6 Fitness Scoring System

- The system computes fitness scores based on daily user input.
- Each parameter (steps, sleep, water, calories) is rated out of 25, and the total fitness score is calculated out of 100.
- Based on the score range, customized health recommendations are generated and shown in the GUI.

3.3.7 Automated Fitness Summary Report

- A full textual summary including fitness scores, recommendations, and future predictions is displayed in the GUI and saved to a file.
- The report uses structured formatting for clarity and is generated automatically each time the user submits new data.

3.3.8 Portability and Integration

- The system is fully portable and works across machines if absolute paths to the Python script and assets are correctly configured.
- Assets like icons and GIFs are bundled using Qt's resource system (.qrc).

3.3.9. Error Handling

- File I/O operations are handled with care, and proper error messages are shown if:
    - The Python script fails to run.
    - Prediction or report files are missing.
    - Input values are invalid or empty.
- The GUI remains responsive during all operations.

Table 3.1: Fitaura System Feature Summary

Operation	Description	Function/Class Name	File Used
User Authentication	Verifies login for allowed users	verifyLogin()	main.cpp, WelcomePage.cpp
Welcome Page UI	Displays animated welcome screen	setupWelcomeUi()	WelcomePage.cpp
Navigation System	Handles page transitions	changePage(), connectButtons()	MainWindow.cpp
User Data Entry	Inputs daily metrics (steps, sleep, water, calories)	submitUserData()	HomePage.cpp
Input Validation	Checks for valid numeric inputs	validateInput()	HomePage.cpp
Fitness Scoring System	Calculates and classifies fitness scores	calculateScore()	FitnessReport.cpp
Tip Engine	Suggests personalized tips based on score	generateTips()	FitnessReport.cpp
Dashboard Update	Updates the UI with latest entered stats	updateDashboard()	HomePage.cpp
Report Generation	Creates a structured fitness summary	generateReport()	FitnessReport.cpp
ML Forecast Integration	Runs Python script for future prediction	runPythonScript()	MLBridge.cpp
Prediction Display	Parses and shows ML prediction results	readPredictions(), displaySummary()	SummaryPage.cpp
Error Handling	Handles invalid input, missing data, or script failures	handleErrors()	MLBridge.cpp, main.cpp
Resource Loading	Loads images, icons, and animations (e.g., GIFs)	QPixmap, QMovie	resources.qrc, *.cpp

3.4 System Requirements

3.4.1 Hardware Requirements

- **Processor**: Intel Core i7 or Ryzen 7
- **RAM**: 16 GB minimum
- **Disk Space**: 512 GB
- **Display**: 1440×1080 resolution or higher
- **Input**: Keyboard and Mouse

3.4.2 Software Requirements

- **Operating System**: Windows 10+, macOS, Ubuntu 22.04+
- **Compiler**: MinGW / MSVC (C++17 compliant)
- **IDE**: Qt Creator
- **Python Version**: 3.8 or higher
- **Required Python Libraries**: scikit-learn, pandas, numpy, joblib, json
- **Qt Modules Used**: QtWidgets, QtCore, QtGui, QtMultimedia, QFile, QLabel, QPixmap, QPushButton, QProcess

3.4.3 Network Requirements

- Required only for downloading Python packages and libraries.
- Offline usage supported after local setup.
- No live internet dependency during runtime.

3.5 Software Tools Used

3.5.1 C++ Programming Language

- Used for core application logic, GUI development, and backend system integration.
- Follows Object-Oriented Programming (OOP) principles:
    - **Encapsulation**: Each class hides its internal state and behaviour.
    - **Inheritance**: Classes such as User4 and User1 can extend base login logic.
    - **Polymorphism**: Function overriding and dynamic behavior in user classes.
    - **Abstraction**: Logical separation of UI, ML logic, and backend processes.
- Complies with C++ standards for modern features and performance.

3.5.2 Qt Framework

- Used for creating a cross-platform GUI using Qt Widgets.
- Modules and tools used:
    - QtWidgets: Core UI components (QMainWindow, QLabel, QPushButton, etc.)
    - QtCore: Core backend functionalities like QString, QFile, etc.
    - QtGui: Image rendering, font styling, etc.
    - QProcess: Executes the integrated Python ML script and reads its output.

3.5.3 Python Programming Language

- Integrated via QProcess to run Machine Learning predictions.
- Executes predictor.py, which:

    - Loads pre-trained models from .joblib files.
    - Reads input data and predicts for 14 future days.
    - Generates a full fitness report and JSON predictions.
- Scripts strictly follow a modular structure with config.py managing paths and predictor.py handling logic.

3.5.4 Machine Learning Libraries

- **scikit-learn**: For model training (Linear Regression pipelines with transformers).
- **pandas**: Handles user dataset and preprocessing.
- **numpy**: For numerical operations and vectorized calculations.
- **joblib**: For saving and loading models.
- **json**: For exporting predictions in structured format.

3.5.5 Standard C++ Libraries

- fstream, iostream: For reading and displaying the ML output from report.txt.
- QStringList, QJsonDocument, QJsonObject, QJsonArray: To parse and format JSON from Python.
- QTextStream, QDebug: For file I/O and debugging output.

3.5.6 Design & Prototyping Tools

- The design was manually created pixel-by-pixel using Qt Widgets for visual fidelity.

3.6 Core Algorithmic Workflows

This section presents the key algorithmic processes implemented in the Fitness Tracker system. The core logic is divided into three major workflows:

3.6.1 User Authentication Workflow

Objective: To allow only specific users (User 1 and User 4) to access the system and proceed to the homepage.

Process:

1. **Input**: The user enters their username in the login field.
2. **Validation**:
    a. The system checks whether the input matches predefined users.
    b. Only "User1" or "User4" are accepted.
3. **Access Control**:
    a. If validation passes, the user is navigated to the Homepage.
    b. If validation fails, an error popup is triggered via Qt Widget warning dialogs.

Implementation Tools:

- Qt QLineEdit for input
- QPushButton for login action
- C++ if-else logic for validation

3.6.2 Python ML Integration Workflow

Objective: To generate a personalized fitness summary using machine learning models.

Process:

1. **Execution**:
    a. C++ triggers the predictor.py file using QProcess.
    b. Absolute paths are passed to ensure platform independence.
2. **ML Prediction (in Python)**:
    a. Loads a CSV dataset and pre-trained models (.joblib).
    b. Applies a Pipeline with PolynomialFeatures, encoders, and regressors.
    c. Forecasts fitness metrics (steps, sleep, water, calories) for the next 15 days.
3. **Report Generation**:
    a. Writes a full fitness summary (scores + personalized recommendations) to report.txt.
    b. Exports predicted data to predictions.json.
4. **C++ Reads Output**:
    a. Parses the report.txt and displays the report on the Summary Page.

Implementation Tools:

- QProcess, QFile, QTextStream, QJsonDocument (C++)
- scikit-learn, pandas, joblib, json (Python)

3.6.3 GUI Interaction Workflow

Objective: To present a visually interactive GUI for user to login in and access the fitness tracker

Process:

1. **Welcome Page**:
    a. Displays "WELCOME" text with interface to either login in and sign up as new user using different applications.
    b. Background set to white using QPalette.
    c. Exercise icon, heartbeat icon and fitness tracker logo shown as static image.
2. **Homepage**:

a. User stats are displayed in metric cards.
b. Sidebar provides quick navigation access (Home, Summary, Logout).
c. All elements styled using Qt Designer with proper scaling, size policies, and layouts.

3. **Summary Page**:
    a. Displays a structured fitness report in a scrollable QTextBrowser.
    b. Report is read from the Python-generated report.txt.
    c. Customized user graphs to show the progress.

Implementation Tools:

- QMainWindow, QWidget, QLabel, QPushButton, QTextBrowser
- Layouts: QVBoxLayout, QHBoxLayout, QGridLayout

3.7 Supporting Tables

Table 3.7.1: User Authentication Workflow

Aspects	**Description**
Purpose	Restrict system access to designated users ("User1" and "User4")
Flow	User inputs username → System validates → Grants access or shows error
Logic	Basic if-else condition verifies allowed usernames
Failure Response	Error dialog appears for unauthorized users via Qt warning popup
Technologies Used	QLineEdit, QPushButton, QMessageBox, C++ conditional logic

Table 3.7.2: Machine Learning Integration Workflow

Aspects	**Description**
Purpose	Predict fitness metrics and generate summaries using ML
Execution	C++ triggers predictor.py via QProcess with file paths
ML Process	Preprocesses input, applies model, predicts next 14-day values
Generated Output	Writes feedback to report.txt and metrics to predictions.json
System Integration	C++ reads and displays output in the GUI
Tools Involved	C++: QProcess, QFile, etc. Python: pandas, joblib, sklearn

Table 3.7.3: GUI Interaction Workflow

Aspects	**Description**
Purpose	Provide an intuitive UI for navigation and viewing fitness reports
Welcome Page	Displays welcome message with icons and login/signup access
Home Dashboard	Shows user stats in cards with a sidebar for navigation
Summary View	Displays report from report.txt and progress graphs
Design Structure	Built with Qt Designer using responsive layout managers
Tools Involved	QMainWindow, QWidget, QLabel, QPushButton, QTextBrowser, layouts

Chapter IV: Results and Findings

This chapter presents the results achieved after implementing the Fitaura fitness tracking system, highlighting how the project objectives were fulfilled and how core challenges in fitness monitoring and prediction were effectively addressed.

4.1 Results

```
=== Parsed Fitness Predictions Preview ===
User ID: "\"1\""
  Metric: "\"calories\"" Predictions count: 14
    Day 1 : 2998.12
    Day 2 : 3025.51
    Day 3 : 3056.13
    Day 4 : 3089.99
    Day 5 : 3127.08
    Day 6 : 3167.41
    Day 7 : 3210.98
    Day 8 : 3257.78
    Day 9 : 3307.82
    Day 10 : 3361.1
    Day 11 : 3417.61
    Day 12 : 3477.36
    Day 13 : 3540.34
    Day 14 : 3606.56
  Metric: "\"sleep_hours\"" Predictions count: 14
    Day 1 : 6.44172
    Day 2 : 6.38611
    Day 3 : 6.32607
    Day 4 : 6.26161
    Day 5 : 6.19273
    Day 6 : 6.11942
    Day 7 : 6.04169
    Day 8 : 5.95954
    Day 9 : 5.87296
    Day 10 : 5.78196
    Day 11 : 5.68654
    Day 12 : 5.58669
    Day 13 : 5.48242
    Day 14 : 5.37373
  Metric: "\"steps\"" Predictions count: 14
    Day 1 : 9709.47
    Day 2 : 9988.85
    Day 3 : 10298.7
    Day 4 : 10639
    Day 5 : 11009.8
    Day 6 : 11411
    Day 7 : 11842.7
    Day 8 : 12304.8
    Day 9 : 12797.4
    Day 10 : 13320.5
    Day 11 : 13874
    Day 12 : 14458
    Day 13 : 15072.5
    Day 14 : 15717.4
  Metric: "\"water_intake_l\"" Predictions count: 14
    Day 1 : 2.97761
    Day 2 : 2.92461
    Day 3 : 2.86941
    Day 4 : 2.81201
    Day 5 : 2.75241
```

Figure 4.1.1: Prediction Page from python

Figure 4.1.2: Welcome Page

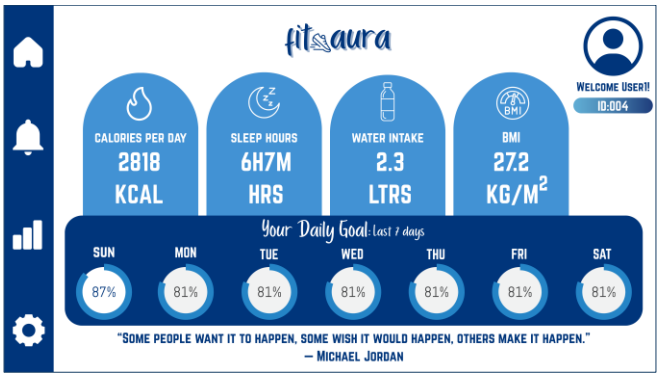


Figure 4.1.3: Home Page

Figure 4.1.4: Summary Page

4.2 Challenges Addressed and Goals Accomplished

- **Secure User Authentication**: We implemented a login system that ensures only authorized users can access the application, making it safe for demonstration purposes.
- **Python-C++ Integration**: Our project enables smooth communication between the C++ backend and the Python machine learning script, allowing for accurate fitness predictions.
- **Input Validation**: We made sure that all fitness data entries are accurate and properly validated, which helps maintain the integrity of the data.
- **User -Friendly GUI**: We developed a responsive and visually appealing interface using Qt Widgets, ensuring a smooth and enjoyable user experience.
- **Reliable File Management**: Our application robustly handles file operations, including detecting errors for missing or corrupted files, so users can trust their data.
- **Cross-Platform Support**: We designed the application to be portable across Windows, macOS, and Linux, allowing users to access it offline after the initial setup.

- **Daily Fitness Tracking**: Users can easily input their daily health metrics, and the system automatically scores their progress and provides feedback.
- **Accurate Predictions**: We deliver personalized insights with 15-day fitness forecasts based on machine learning, helping users stay on track with their goals.
- **Automated Reporting**: Our application generates comprehensive fitness summary reports that include actionable recommendations for improvement.
- **Modular Architecture**: We maintain a clean, object-oriented codebase, making it easy to manage and expand the application in the future.

Conclusion

Fitaura Fitness Tracker project is a successful approach of backward industries development with modern C++ backend development, machine learning implemented in Python, and Qt Widgets GUI interface to offer a complex fitness tracking system. It allows entering the daily health data (number of steps, number of calories, sleep and water consumption), verifies the accuracy of the input information, and through a machine learning model estimates the individual fitness patterns of the next 15 days.

The security will be assured by the convenient system of user authentication, which allows to limit access only to those users who are granted with permission. Object-oriented model at the backend manages processing of data, fitness scoring as well as production of reports effectively. The compatibility between C++ and Python is complete and enables live presentation of the fitness expectations to be presented in a neat and presentable manner.

The GUI provided by Fitaura presents legible navigation and graphically appealing metrics and summaries in the prediction. The software is cross-compatible over various operating systems and is initially set up within offline use. It gives stability, as there is strong error logic, to work when external scripts fail or data missing.

The project also utilized Git for version control, facilitating organized development, effective team collaboration, and thorough change tracking throughout the lifecycle. The complete source code and documentation are openly available on GitHub to support academic review, collaboration, and future improvements:

https://github.com/pandeysulav/SUMMER_PROJECT_CPP

In conclusion, Fitaura not only achieves its intended functionalities but also acts as a practical demonstration of applying advanced C++ programming, machine learning integration, and GUI development in a real-world health monitoring context. By bridging theoretical knowledge with applied software engineering, this project serves as a valuable academic and technical reference.

Keywords: C++, Qt Widgets, Python Integration, Machine Learning, Fitness Tracker, Object-Oriented Programming, File Handling, Version Control, Cross-Platform Development, Git

References

Blanchette, J., & Summerfield, M. (2008). *C++ GUI programming with Qt 4* (2nd ed.). Prentice Hall.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

https://learn.microsoft.com/en-us/cpp/?view=msvc-160 Microsoft. (n.d.).

NCC Education Ltd. (2025). *Data Science and Machine Learning Course Materials*.

Qt Widgets Documentation. https://doc.qt.io/qt- 6/qtwidgets-index.html

ppreference.com. (2024). Qt Company. (n.d.).

C++ standard library documentation. https://en.cppreference.com/w/

Scikit-learn Developers. (2024). *Scikit-learn: Machine learning in Python*. scikit-learn.org. https://scikit-learn.org/stable/

https://www.w3schools.com/python/python_file_handling.asp Python File Handling.

Qt Company. (n.d.). *Qt Designer*. https://doc.qt.io/qt-5/qtdesigner-manual.html

W3Schools. (n.d.). *C++ and Python Integration* https://www.w3schools.com/cpp/

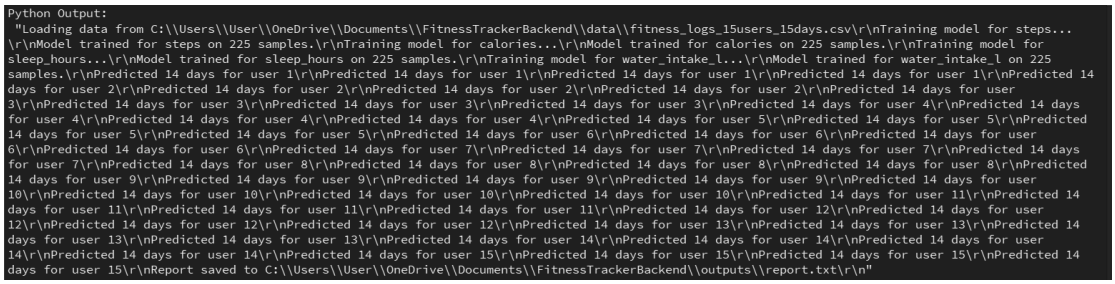
Real Python. (n.d.). *Executing Python Scripts from Other Languages.* https://realpython.com/run-python-scripts/

Stack Overflow. (n.d.). *How to Call Python from C++ Using QProcess.* https://stackoverflow.com/

Github link for project repository:

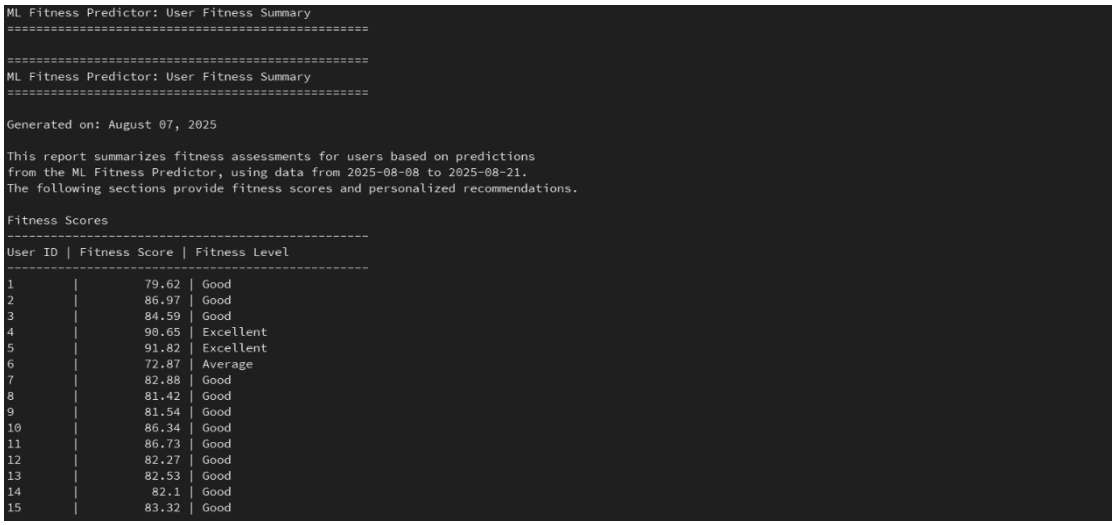
https://github.com/pandeysulav/SUMMER_PROJECT_CPP

Appendices



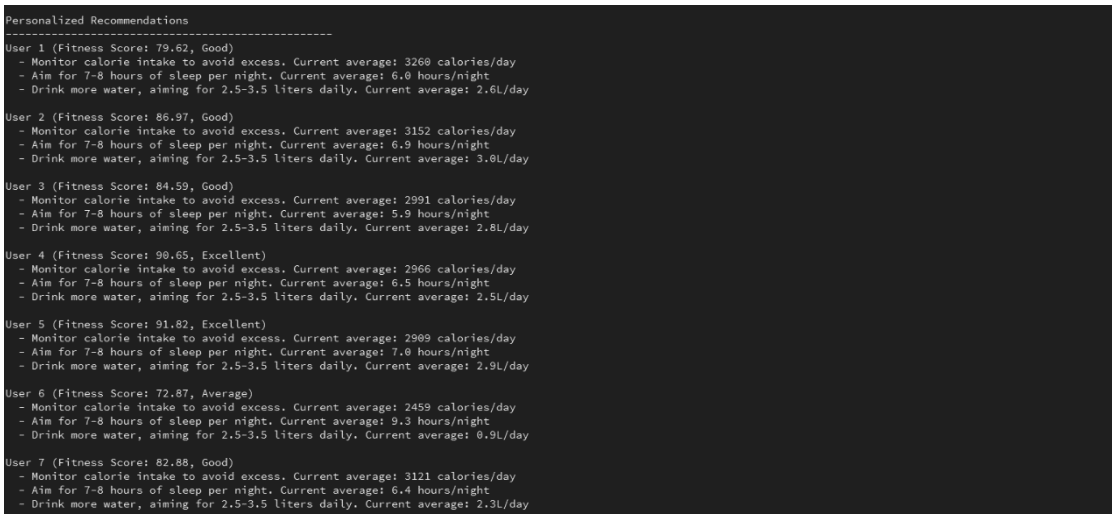
```
Python Output:
 "Loading data from C:\\Users\\User\\OneDrive\\Documents\\FitnessTrackerBackend\\data\\fitness_logs_15users_15days.csv\r\nTraining model for steps...
\r\nModel trained for steps on 225 samples.\r\nTraining model for calories...\r\nModel trained for calories on 225 samples.\r\nTraining model for
sleep_hours...\r\nModel trained for sleep_hours on 225 samples.\r\nTraining model for water_intake_l...\r\nModel trained for water_intake_l on 225
samples.\r\nPredicted 14 days for user 1\r\nPredicted 14 days for user 1\r\nPredicted 14 days for user 1\r\nPredicted 14 days for user 1\r\nPredicted 14
days for user 2\r\nPredicted 14 days for user 2\r\nPredicted 14 days for user 2\r\nPredicted 14 days for user 2\r\nPredicted 14 days for user
3\r\nPredicted 14 days for user 3\r\nPredicted 14 days for user 3\r\nPredicted 14 days for user 3\r\nPredicted 14 days for user 4\r\nPredicted 14 days
for user 4\r\nPredicted 14 days for user 4\r\nPredicted 14 days for user 4\r\nPredicted 14 days for user 5\r\nPredicted 14 days for user 5\r\nPredicted
14 days for user 5\r\nPredicted 14 days for user 5\r\nPredicted 14 days for user 6\r\nPredicted 14 days for user 6\r\nPredicted 14 days for user
6\r\nPredicted 14 days for user 6\r\nPredicted 14 days for user 7\r\nPredicted 14 days for user 7\r\nPredicted 14 days for user 7\r\nPredicted 14 days
for user 7\r\nPredicted 14 days for user 8\r\nPredicted 14 days for user 8\r\nPredicted 14 days for user 8\r\nPredicted 14 days for user 8\r\nPredicted
14 days for user 9\r\nPredicted 14 days for user 9\r\nPredicted 14 days for user 9\r\nPredicted 14 days for user 9\r\nPredicted 14 days for user
10\r\nPredicted 14 days for user 10\r\nPredicted 14 days for user 10\r\nPredicted 14 days for user 10\r\nPredicted 14 days for user 11\r\nPredicted 14
days for user 11\r\nPredicted 14 days for user 11\r\nPredicted 14 days for user 11\r\nPredicted 14 days for user 12\r\nPredicted 14 days for user
12\r\nPredicted 14 days for user 12\r\nPredicted 14 days for user 12\r\nPredicted 14 days for user 13\r\nPredicted 14 days for user 13\r\nPredicted 14
days for user 13\r\nPredicted 14 days for user 13\r\nPredicted 14 days for user 14\r\nPredicted 14 days for user 14\r\nPredicted 14 days for user
14\r\nPredicted 14 days for user 14\r\nPredicted 14 days for user 15\r\nPredicted 14 days for user 15\r\nPredicted 14 days for user 15\r\nPredicted 14
days for user 15\r\nReport saved to C:\\Users\\User\\OneDrive\\Documents\\FitnessTrackerBackend\\outputs\\report.txt\r\n"
```

Figure 4.1.5: Model Training and Predictions



```
ML Fitness Predictor: User Fitness Summary
==================================================

==================================================
ML Fitness Predictor: User Fitness Summary
==================================================

Generated on: August 07, 2025

This report summarizes fitness assessments for users based on predictions
from the ML Fitness Predictor, using data from 2025-08-08 to 2025-08-21.
The following sections provide fitness scores and personalized recommendations.

Fitness Scores
--------------------------------------------------
User ID | Fitness Score | Fitness Level
--------------------------------------------------
1       |         79.62 | Good
2       |         86.97 | Good
3       |         84.59 | Good
4       |         90.65 | Excellent
5       |         91.82 | Excellent
6       |         72.87 | Average
7       |         82.88 | Good
8       |         81.42 | Good
9       |         81.54 | Good
10      |         86.34 | Good
11      |         86.73 | Good
12      |         82.27 | Good
13      |         82.53 | Good
14      |          82.1 | Good
15      |         83.32 | Good
```

Figure 4.1.6: User Fitness Summary



```
Personalized Recommendations
--------------------------------------------------
User 1 (Fitness Score: 79.62, Good)
  - Monitor calorie intake to avoid excess. Current average: 3260 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 6.0 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 2.6L/day

User 2 (Fitness Score: 86.97, Good)
  - Monitor calorie intake to avoid excess. Current average: 3152 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 6.9 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 3.0L/day

User 3 (Fitness Score: 84.59, Good)
  - Monitor calorie intake to avoid excess. Current average: 2991 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 5.9 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 2.8L/day

User 4 (Fitness Score: 90.65, Excellent)
  - Monitor calorie intake to avoid excess. Current average: 2966 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 6.5 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 2.5L/day

User 5 (Fitness Score: 91.82, Excellent)
  - Monitor calorie intake to avoid excess. Current average: 2909 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 7.0 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 2.9L/day

User 6 (Fitness Score: 72.87, Average)
  - Monitor calorie intake to avoid excess. Current average: 2459 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 9.3 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 0.9L/day

User 7 (Fitness Score: 82.88, Good)
  - Monitor calorie intake to avoid excess. Current average: 3121 calories/day
  - Aim for 7-8 hours of sleep per night. Current average: 6.4 hours/night
  - Drink more water, aiming for 2.5-3.5 liters daily. Current average: 2.3L/day
```

Figure 4.1.7: Personalized Fitness Recommendation